

实验一

题目一

给定长度为 n 且无重复元素的有序数组 $nums$ ，以及目标值 t 。在 $nums$ 中找到 t ，并返回其索引（从 0 开始）。如果 t 不存在于 $nums$ 中，返回它将会被按顺序插入的位置。算法时间复杂度应为 $O(\log n)$ 。

题目二

给定两个整数数组 a 和 b ，长度分别为 n 和 m ，给出两个数组大小的判定算法。如果对于任意 i ， $0 \leq i < j$ ， $a_i = b_i$ ，存在 j 使得 $a_j < b_j$ ，则称 $a < b$ 。或者，对任意 i ($0 \leq i < n$)，有 $a_i = b_i$ ，且 $n < m$ ，则称 $a < b$ 。对任意 i ($0 \leq i < n$)，有 $a_i = b_i$ ，且 $n = m$ ，则称 $a = b$ 。

题目三

```
1 SparseMatrix SparseMatrix::FastTranspose()
2 { // Return the transpose of *this in O(terms+cols) time.
3   SparseMatrix b(cols, rows, terms);
4   if (terms > 0)
5   { // nonzero matrix
6     int *rowSize = new int[cols];
7     int *rowStart = new int[cols];
8     // compute rowSize[i] = number of terms in row i of b
9     fill(rowSize, rowSize + cols, 0); // initialize
10    for (i = 0; i < terms; i++) rowSize[smArray[i].col]++;

11    // rowStart[i] = starting position of row i in b
12    rowStart[0] = 0;
13    for (i = 1; i < cols; i++) rowStart[i] = rowStart[i-1] + rowSize[i-1];

14    for (i = 0; i < terms; i++)
15    { // copy from *this to b
16      int j = rowStart[smArray[i].col];
17      b.smArray[j].row = smArray[i].col;
18      b.smArray[j].col = smArray[i].row;
19      b.smArray[j].value = smArray[i].value;
20      rowStart[smArray[i].col]++;
21    } // end of for
22    delete [] rowSize;
23    delete [] rowStart;
24  } // end of if
25  return b;
26 }
```

Program 2.11: Transposing a matrix faster

矩阵转置，重写 FastTranspose，要求只用一个数组保存 RowSize 和 RowStart 的信息。时间复杂度 $O(\text{terms} + \text{cols})$ 。

题目四

获得函数 BinarySearch 的平均运行时间。在 $[0, 100]$ 范围内选取合适的 n 值。报告需

包括实验计划、测得时间（表格和图形）。